when they see a pointer-to-integer or an integer-to-pointer cast, they tend to bail out. By introducing a new instruction for pointer subtraction, we are able to reduce the number of these casts significantly (as shown later in the evaluation).

4.8 Memory Block Lifetime

Besides the `p + n == q` case, the machine addresses of different logical pointers may also be equal when addresses get reused by the allocator. For example:

```
char *p = malloc(4);
free(p);
char *q = malloc(4);
if (p == q) { /* ... */ }
```

We would again like to optimize the comparison to `false` because we are comparing the results of two separate calls to `malloc`. And again, in an actual execution, the addresses may be equal because `malloc` can reuse memory after `free`.

A common solution to this problem is to let the behavior of pointer equality depend on whether the block that `p` points to is still allocated. However, the problem with that approach is that it makes comparison not freely reorderable with deallocation. Another solution is to define comparisons with freed blocks as UB (as C and C++ standards do), but that limits movement of pointer comparisons as well. Instead, we add the concept of a *lifetime* to our memory blocks. The memory time stamp gets incremented on every memory allocation and deallocation. The end of the lifetime of a block is initially ∞ and gets set when the block is deallocated.

In the above situation, because the lifetimes of the two blocks do not overlap, we again make pointer comparison non-deterministic, justifying the optimization. This is reflected in ICMP-PTR-LOGICAL-NONDET-FALSE, which also applies if the lifetimes of the blocks are disjoint.

This lifetime-based handling of pointer comparison has the caveat that a `free` can no longer be moved up above a `malloc` of a different block, since that would make the two blocks' lifetimes no longer overlap, possibly affecting program behavior. Although this an interesting optimization to support (to reduce peak memory consumption and potentially reuse cached memory), LLVM does not perform it.

4.9 Load and Store

To perform a memory access of size $sz > 0$ through a pointer p on memory M, the pointer must be *dereferenceable*, written $\mathsf{deref}_M(p, sz)$. If p is a logical pointer $\mathsf{Log}(l, o, s)$ where block l is not freed and $\mathsf{inbounds}_M(l, o) \wedge \mathsf{inbounds}_M(l, o + sz)$, then we have $\mathsf{deref}_M(p, sz)$.

If p is a physical pointer $\mathsf{Phy}(o, s, I, cid)$, there must be a still-alive block $(t, (b, \infty), n, a, c, P)$ with id l and an offset o_l such that $P(s)_0 + o_l = o$ and $\mathsf{inbounds}_M(l, o_l) \wedge \mathsf{inbounds}_M(l, o_l + sz)$. (Note that l and o_l are uniquely determined since memory blocks are disjoint and $sz > 0$.) Moreover, all addresses $o' \in I$ must be inbounds of the same block, i.e., $\forall o' \in I, \mathsf{inbounds}_M(l, o' - P(s)_0)$. If the pointer was derived from a parameter (i.e., $cid \neq$ **None**) and the function corresponding to that parameter did not return yet (i.e., $M(cid) \neq$ **None**), then $b < M(cid)$, i.e., the block must have been allocated before the function call identified by cid started. If all these requirements are satisfied, we have $\mathsf{deref}_M(p, sz)$.

To support conversion between values and low-level bitwise representation, we define two meta operations, $ty\Downarrow \in ([\![ty]\!] \to \mathrm{Bit}^{\mathrm{bitwidth}(ty)})$ and $ty\Uparrow \in (\mathrm{Bit}^{\mathrm{bitwidth}(ty)} \to [\![ty]\!])$. For base types, $ty\Downarrow$ transforms **poison** into the bitvector of all **poison** bits, and defined values into their standard low-level representation (Fig. 6a). getbit $v\ i$ is a partial function that returns the ith bit of a value v. If v is a pointer, getbit $v\ i$ returns either **poison** if v is **poison** or a pair (p, i) which is an element